# Dron-Can: Feasibility of Aerial Detection of Lost Dogs Using Commercial UAVs and Computer Vision

Max Gabriel Sánchez Flores
*Facultad de Ingeniería*
*Universidad Evangélica*
*de El Salvador*
San Salvador, El Salvador
maxg.sanchezf@gmail.com
ORCID: 0009-0008-5414-2289

Jhoan Daniel Martínez Marroquín
*Facultad de Ingeniería*
*Universidad Evangélica*
*de El Salvador*
San Salvador, El Salvador
jh3027d@gmail.com
ORCID: 0009-0001-6100-3075

Óscar Eduardo Díaz Gómez
*Facultad de Ingeniería*
*Universidad Evangélica*
*de El Salvador*
San Salvador, El Salvador
ed28356@gmail.com
ORCID: 0009-0008-4167-1755

***Abstract*—Dron-Can is an exploratory proof of concept combining video acquired with a commercial DJI Mavic Mini drone, a YOLO11s detector, and a local interface for offline inference. The study documents an implemented pipeline and examines its archived development evidence from recordings of one Schnauzer in a local park. The final dataset contains 3,882 images, including augmented derivatives: 3,309 training, 358 validation, and 215 test images. The historical ten-epoch run reports internal validation precision of 0.99671, recall of 0.99443, mAP at IoU 0.5 of 0.99480, and mAP across IoU 0.5 to 0.95 of 0.81384. An audit identifies shared augmentation-origin families across partitions and a common source-video prefix, preventing interpretation of these metrics as independent estimates for new dogs or environments. The results support the feasibility of the offline integration within the recorded setting. Live transmission, breed discrimination, individual identification, and operational search effectiveness require separate validation.**

***Keywords—computer vision, aerial imagery, dog detection, YOLO11, unmanned aerial vehicles, internal validation***

## I. INTRODUCTION

Aerial observation can provide viewpoints that are difficult to obtain from the ground and can support the visual inspection of an area. For animal detection, however, the usefulness of those viewpoints depends on image resolution, scene conditions, target visibility, and the evaluation protocol. Reviews of automated wildlife detection from drone imagery describe opportunities alongside constraints associated with species and survey conditions [1]. Detecting an animal in an image is only one component of a search process; it does not establish that the animal is lost or identify its owner.

Dron-Can originated as a course project integrating aerial recording, manual frame selection and annotation, adaptation of a pretrained detector to one class, and a desktop interface for prerecorded videos. Its contribution is a documented implementation workflow and an examination of the evidence supporting it. The study evaluates an accessible offline integration rather than a new detection architecture or autonomous flight system.

The question addressed is whether these components can form a working offline prototype for inspecting dog-containing aerial footage. Reliability in searches involving new animals and locations is a separate question requiring independent evaluation. The recorded material supports the integration question within its restricted setting.

The analysis combines the development description with archived datasets, configurations, a training log, saved figures, and a notebook. It examines partition dependencies and distinguishes prediction on test images from quantitative testing. Accordingly, the results are reported as historical development evidence, with explicit boundaries on their interpretation.

### *A. Related Work and Positioning*

Animal detection and counting from UAV imagery have been studied using convolutional neural networks [2]. Their evaluation setting differs from the present single-dog prototype, so their results are not used as directly comparable benchmarks. Pet identification is also a separate task: Azizi and Zaman combine face, body, and other information to match individual animals [3]. Dron-Can uses one detection label and does not implement those identity-matching stages.

WAID provides a public aerial wildlife dataset for developing detection methods [4]. Its relevance here is the need for representative aerial observations, rather than evidence that a dog detector trained on one local recording will transfer to other animals. YOLO11 supplies an established model family and software implementation [5]. Our use of that software is an integration choice; neither the availability of a detector nor strong scores on a restricted validation set establishes operational readiness.

Evaluation must also account for dependencies among observations. Leakage can arise when information shared with training data enters an evaluation partition [6]. For video-derived images, different filenames do not necessarily indicate different scenes or acquisition events. Augmentation increases the number of training examples but does not create independent animals, sites, or flights. These considerations determine how the evidence below should be interpreted.

## II. MATERIALS AND METHODS

### *A. Acquisition and Implemented Workflow*

The acquisition platform was a DJI Mavic Mini, operated manually to record the study subject, a Schnauzer named Nico, in a local park. The drone served as the camera platform. Fig. 1 illustrates the available acquisition setting. Recorded footage includes a dog and a person; their co-occurrence does not constitute a labeled behavioral or lost-status task. No comparison with a phone or ground camera was performed, and no optimal altitude was established experimentally. Exact flight

trajectories and a complete flight-level acquisition log are not available for this analysis.



Fig. 1. Existing aerial image from the local acquisition setting. The study concerns one dog; the image illustrates the recording context and is not an independent field-evaluation result.

The implemented sequence was aerial recording, frame extraction and selection, bounding-box annotation, dataset preparation and augmentation, supervised fine-tuning, internal validation, and offline video inference. Roboflow was used for annotation and dataset export; the development materials also describe Keras-based image augmentation. Training used the Ultralytics implementation in Google Colab. The desktop application combined a Tkinter-based interface with OpenCV for video handling and the detector for frame-level inference. Acquisition and inference were separate stages: the application did not receive a live drone video stream.

### *B. Dataset Versions and Annotation*

The archived Roboflow exports contain dataset versions 1 and 2 [7]. These are dataset identifiers and must be distinguished from the development model labels v1, v2, and v2.1 used by the team. Table I reports the exact image and annotation counts recovered from the exports. Both dataset configuration files specify one class, named schnauzer, with class index 0. This name is the training label; it does not demonstrate discrimination among dog breeds.

TABLE I. ARCHIVED DATASET PARTITIONS

| Dataset | Partition | Images | Boxes |
|---|---|---|---|
| 1 | Train | 200 | 200 |
| 1 | Validation | 100 | 100 |
| 1 | Test | 86 | 86 |
| 2 | Train | 3,309 | 3,309 |
| 2 | Validation | 358 | 359 |
| 2 | Test | 215 | 215 |

Dataset 1 contains 386 images. Its export metadata specify automatic orientation and stretching to 640 by 640 pixels, with no Roboflow augmentation at export. Dataset 2 contains 3,882 images, with the same output size and three versions of training source images. Listed transforms are horizontal flipping (50%), rotation (up to 15 degrees), horizontal and vertical shear (up to 10 degrees), brightness (up to 15%) and exposure (up to 10%) changes, Gaussian blur (up to 2.5 pixels), and salt-and-pepper noise (0.1%). These are exported examples, not distinct captured scenes.

A separate Keras image archive contains 1,158 JPEGs: three indexed derivatives for each of 386 filename-origin identifiers. It has no accompanying labels or split manifest and is not the final 3,882-image detector dataset. In dataset 2, filenames marked as Keras derivatives occur in all three partitions, including validation and test. The Roboflow export metadata alone therefore do not establish that all augmentation was restricted to training or that the partitions were assigned before the earlier augmentation stage.

The annotation audit found an image-label pair for each image, valid five-field YOLO rows with class index 0, normalized coordinates within range, and positive box dimensions. Dataset 2 validation contains one more box than images because one image has two annotations. No empty label files were found in either export. Consequently, the available partitions do not include a documented set of dog-free images for measuring false alarms in scenes without the target. Annotation-format consistency also does not establish annotation accuracy: no independent second-annotator agreement measurement was recorded.

### *C. Partition Dependencies*

We audited the existing partitions without changing them.

Images with identical byte content were compared using SHA-256, and filename-origin groups were examined after removing the Roboflow export hash. For Keras-derived names of the form dog_aug_NNN_VV, the numeric origin NNN was retained and the derivative index VV was removed. This grouping provides evidence about shared augmentation origins; it is not a complete reconstruction of the original video chronology.

No byte-identical cross-partition images were found in either export. Nevertheless, dataset 2 has 171 shared Keras origin families between training and validation, 98 between training and test, and 38 between validation and test. These are pairwise family counts, not numbers of overlapping images, and the sets can intersect. Inspection of one family distributed across all three partitions confirmed visually related derivatives of the same scene. The source prefix Comp-1 also occurs across all partitions in both exports, so different filenames do not establish separation by recording.

The audit identifies cross-partition dependence and a leakage risk that exact-duplicate checking would miss. It does not quantify the resulting inflation of precision, recall, or average precision. Independence at the dog, video, flight, and environment levels has not been established. The current partitions are retained only to describe the historical run; they are not presented as a clean benchmark for generalization.

### *D. Development Versions and Training Configuration*

The development record describes model versions v1, v2, and v2.1, with changes to frame selection, augmentation, and training duration. The final archived run is associated with dataset 2 and model v2.1. Exact export counts take precedence over approximate totals in the earlier narrative. Because the changes were not isolated and comparable logs are incomplete, the record does not establish the effect of any single factor.

Table II summarizes the historical final-run configuration.

The saved checkpoint is dated October 28, 2025 and records Ultralytics 8.3.221, YOLO11s, and dataset 2. Its stored training-

results series agrees with the ten rows of the archived CSV, supporting the provenance of the reported metrics.

TABLE II. HISTORICAL FINAL-RUN CONFIGURATION

| Setting | Archived value |
|---|---|
| Detector / initialization | YOLO11s / pretrained weights |
| Software | Ultralytics 8.3.221 |
| Dataset / validation split | Roboflow version 2 / val |
| Epochs / batch size | 10 / 16 |
| Input size | 640 by 640 pixels |
| Optimizer | Automatic selection |
| Seed / deterministic flag | 0 / enabled |
| Mixed precision | Enabled |
| NMS IoU threshold | 0.70 |

The implementation uses pretrained initialization, dataset preparation, and ten-epoch fine-tuning with existing software. The optimizer was selected automatically; its selection was not evaluated independently. The saved seed and deterministic flag describe the run, but one run cannot establish variability across initializations. The nonmaximum suppression (NMS) IoU threshold controls removal of overlapping predictions and differs from the IoU thresholds used to score localization.

### *E. Metrics and Offline Application*

Precision describes the proportion of reported detections that match labeled targets, while recall describes the proportion of labeled targets recovered. Intersection over union (IoU) measures the overlap between a predicted box and its reference box. Average precision integrates the precision-recall relationship. With one class, mAP at IoU 0.5 is that class's average precision, and mAP across IoU 0.5 to 0.95 averages average precision over ten IoU thresholds separated by 0.05. These are detection metrics, not image-classification accuracy [8].

The archived training CSV contains internal validation metrics at each epoch. In Ultralytics 8.3.221, summary precision and recall are selected from the confidence curves at the maximum mean F1 operating point [8]. The confusion-matrix calculation uses a separate matching rule [8], [9]. Matrix counts and CSV precision therefore need not give identical precision estimates. The interface confidence slider likewise sets a prediction filter; a displayed percentage is not an aggregate accuracy measurement.

The application loads a local model and video file, reads successive frames, runs inference, and displays boxes, class labels, and confidence scores. Users can select model versions, adjust the confidence threshold and inference image size, and pause or resume playback. Fig. 2 shows the final existing interface. These controls support exploration of prerecorded footage. The application does not identify an individual dog, infer whether it is accompanied, geolocate a detection, or direct an autonomous search.

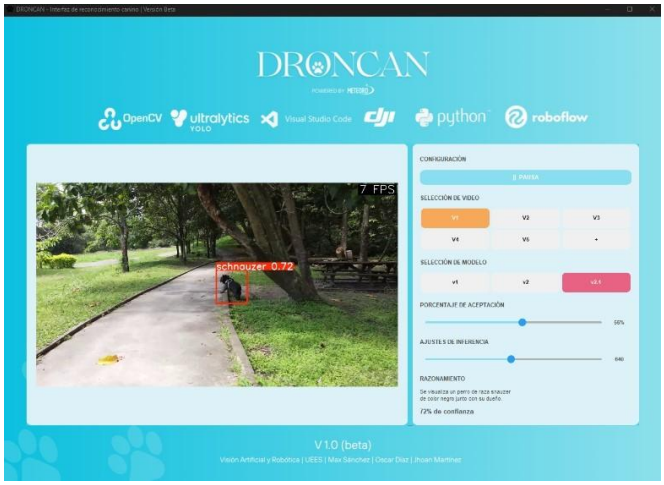



Fig. 2. Existing desktop interface displaying offline predictions on a recorded video. Confidence and FPS overlays belong to the shown execution; they are not aggregate accuracy or latency benchmarks.

## III. RESULTS

### *A. Historical Internal Validation*

The final row of the historical ten-epoch CSV reports precision 0.99671, recall 0.99443, mAP at IoU 0.5 of 0.99480, and mAP across IoU 0.5 to 0.95 of 0.81384. These values refer to the internal validation partition associated with the final archived run, comprising 358 images and 359 labeled boxes. They are not quantitative results from the 215-image test partition or from an independent field trial. The partition dependencies described in Section II-C apply to all interpretations of these scores.

Fig. 3 reproduces the CSV values without additional smoothing. Precision rises from 0.73594 at epoch 1 to 0.99671 at epoch 10; recall rises from 0.76856 to 0.99443. The mAP at IoU 0.5 increases from 0.76178 to 0.99480, whereas the more demanding mAP across IoU thresholds increases from 0.41115 to 0.81384. Recall and average precision fluctuate at intermediate epochs rather than increasing monotonically. The lower score under stricter IoU matching indicates a difference in localization criteria within this dataset, not measured adaptation to other breeds or sites.

Training box loss decreases from 1.50277 to 1.02681, and validation box loss decreases overall from 1.55459 to 0.80309. Validation loss has an intermediate increase at epoch 6. The curves describe optimization within one historical run; the dependence between partitions prevents using their shape alone to establish external generalization. Ten epochs and high scores are not, by themselves, a diagnosis of overfitting or evidence that overfitting has been avoided.

### *B. Archived Confusion Matrix*

Fig. 4 redraws the archived raw-count matrix containing 357 matched target detections, four unmatched predictions, and two unmatched labeled targets. Its 359 target annotations are consistent with dataset 2 validation. The association relies on the saved artifact and its label count; a separate matrix-specific execution log is not available. In the archived software version, the default matrix calculation applies confidence 0.25 and

matching IoU 0.45 [8], [9]. These defaults provide context, but are not asserted as independently recovered runtime measurements for this figure.

The background row and column encode unmatched objects in an object-detection matrix. They do not supply the number of correctly rejected dog-free images. Thus, specificity and a background false-alarm rate cannot be calculated from this plot. Its displayed counts imply precision of approximately 0.98892 and recall of 0.99443 at the matrix operating point. The difference from CSV precision is compatible with the library's distinct confidence and matching rules; the matrix is not used to replace the CSV series or claim an additional experiment.

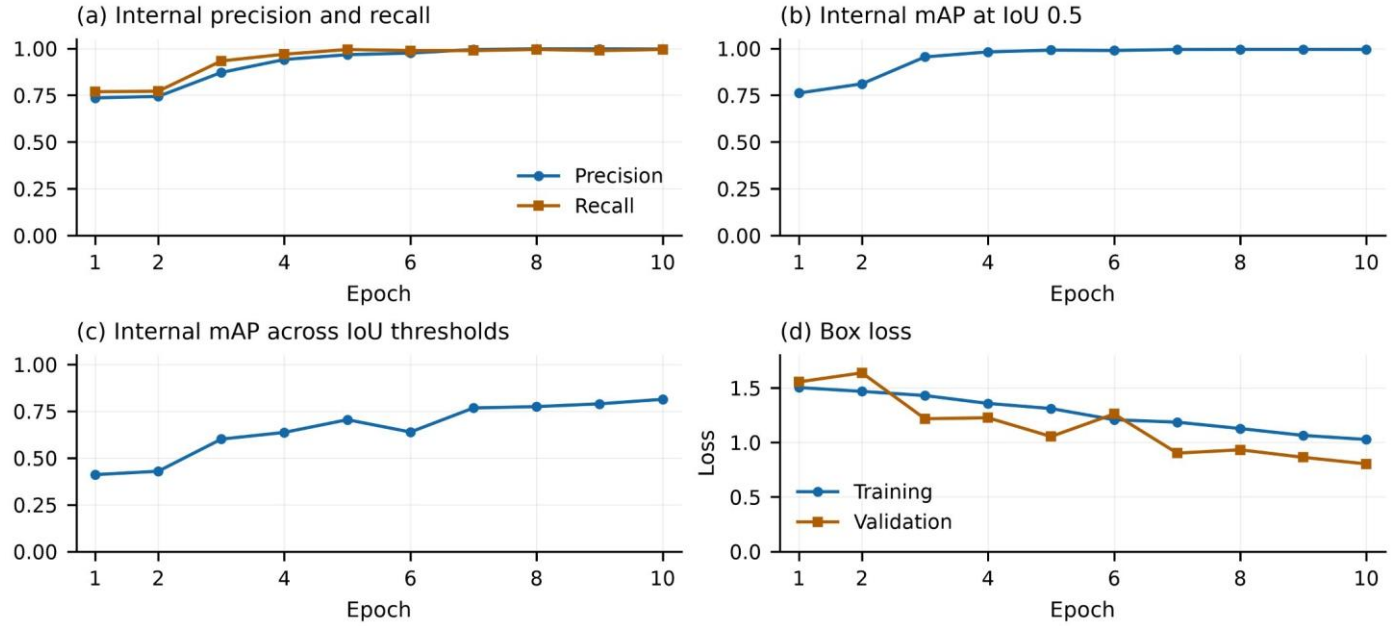



Fig. 3. Archived ten-epoch training and internal validation series: (a) precision and recall, (b) mAP at IoU 0.5, (c) mAP across IoU 0.5 to 0.95, and (d) box losses. Points are the original CSV values, joined without additional smoothing. Shared augmentation origins across partitions limit their interpretation as estimates for new observations.

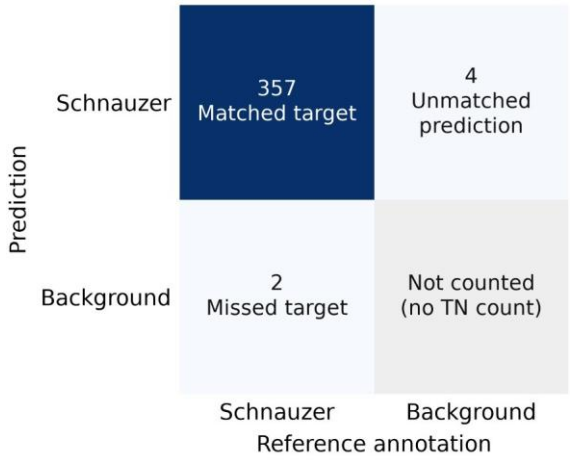



Fig. 4. Counts transcribed from the archived detection matrix: 357 matched detections, 4 unmatched predictions, and 2 missed targets. Rows denote predictions and columns denote reference annotations. The lower-right cell has no true-negative count. Operating thresholds are not independently documented for this saved plot.

### *C. Functional Evidence and Notebook Provenance*

The saved interface image demonstrates that the application can overlay detector output on a prerecorded video. It is illustrative functional evidence, with no independently specified video-level test protocol. FPS values in individual screenshots do not provide a latency distribution, hardware benchmark, or drone-to-display response time. The available evidence does not measure coverage, search duration, missed dogs during complete flights, or the probability of recovering a lost animal.

The shared notebook also contains a call that predicts on the test-image directory and saved outputs for 215 images. This is qualitative inference, not a labeled quantitative test evaluation: no saved test-partition validation report is documented.

The shared notebook also contains outputs from Ultralytics 8.4.174 with a different mAP across IoU thresholds, approximately 0.806. Those outputs belong to a separate run and are not merged with the historical 8.3.221 CSV or checkpoint. Keeping these records separate preserves the provenance of the reported results.

## IV. DISCUSSION AND LIMITATIONS

### *A. What the Evidence Establishes*

The prototype implements a complete offline path from acquired video to annotated frames, fine-tuning, and visualized predictions. This is a practical feasibility result at the development stage. Its robotics component is operator-controlled aerial acquisition. The work does not establish autonomy, live communication, onboard computation, or a comparative advantage over terrestrial recording. The potential value of aerial viewpoints remains a motivation that would require a separate search-oriented evaluation.

The numerical results establish that the historical model fits and detects the single labeled target well within the available internal validation material. The domain is narrow: one subject, one acquisition setting, no documented target-free evaluation images, and no verified independence by source recording. Augmentation can change the appearance of existing samples, but its many derivatives are not equivalent to many independent dogs or environments. Strong internal metrics can therefore coexist with weak evidence for the intended application.

The shared filename-origin families make the independence concern concrete: removing byte-identical duplicates alone would leave transformed relatives in different partitions. The archived artifacts do not quantify the effect on the metrics, and the loss curves cannot distinguish memorization from transferable features under this split. Independent observations are therefore needed to evaluate generalization.

### *B. Lessons From Iterative Development*

Frame selection, annotation, augmentation, and training settings changed during development, motivating further curation and interface inspection. These practical observations were not tested in an ablation study. They do not establish that data quality caused a measured improvement over quantity, that ten epochs were optimal, or that the final version is statistically superior under a common protocol.

The resulting reproducibility lesson is to retain acquisition identifiers, augmentation lineage, software versions, split manifests, and evaluation outputs. The historical checkpoint and CSV support recovery of the final reported metrics, whereas the changing notebook illustrates why a shared notebook alone is not a stable experiment record. Basic annotation-format checks are also necessary but insufficient: they cannot establish source independence, label correctness, or field reliability.

### *C. Future Validation*

A defensible future evaluation would separate dogs, source videos, sessions, and locations before producing training augmentations. All derivatives of a source observation would remain in its assigned group, and an untouched evaluation set would include both target-present and target-absent footage. Multiple dogs and environments are needed to assess transfer. The current exports cannot create that diversity by rearranging files or applying more transformations; independent acquisition would still be required.

Once independent observations are available, fixed evaluation rules should specify model selection, confidence filtering, IoU matching, and the unit of analysis. Reporting errors by recording, target size, visibility, and environment would help determine where the pipeline fails. Repeated runs and uncertainty estimates would be useful, provided that dependence between nearby frames is handled rather than treating every derivative as an independent sample. These are proposed validation steps, not completed results.

Live transmission would require separate implementation and measurement of communication stability, processing latency, and the complete acquisition-to-display path. Operational searches would also need a defined procedure and human verification of detections. These are proposed extensions to the demonstrated offline workflow, not capabilities established by the present evidence.

## V. CONCLUSION

Dron-Can demonstrates an implemented exploratory pipeline for offline processing of aerial dog videos with YOLO11s and a local interface. The historical run provides traceable internal validation results within the recorded setting. Shared augmentation origins, a common acquisition context, and the absence of independent quantitative field testing limit their interpretation to development evidence. The study contributes a documented integration workflow and identifies the validation steps needed to assess its intended application: independent observations across dogs and environments, target-free footage, and measured live integration.

## REFERENCES


[1] E. Corcoran, M. Winsen, A. Sudholz, and G. Hamilton, "Automated detection of wildlife using drones: Synthesis, opportunities and constraints," Methods Ecol. Evol., vol. 12, no. 6, pp. 1103–1114, 2021, doi: 10.1111/2041-210X.13581.

[2] K. Rančić et al., "Animal detection and counting from UAV images using convolutional neural networks," Drones, vol. 7, no. 3, Art. no. 179, 2023, doi: 10.3390/drones7030179.

[3] E. Azizi and L. Zaman, "Deep learning pet identification using face and body," Information, vol. 14, no. 5, Art. no. 278, 2023, doi: 10.3390/info14050278.

[4] C. Mou, T. Liu, C. Zhu, and X. Cui, "WAID: A large-scale dataset for wildlife detection with drones," Appl. Sci., vol. 13, no. 18, Art. no. 10397, 2023, doi: 10.3390/app131810397.

[5] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024. [Online]. Available: https://docs.ultralytics.com/models/yolo11/ (accessed Oct. 8, 2026). Historical software release: https://github.com/ultralytics/ultralytics/releases/tag/v8.3.221.

[6] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," Patterns, vol. 4, no. 9, Art. no. 100804, 2023, doi: 10.1016/j.patter.2023.100804.

[7] Dron-Can project dataset, "NicoDogDetection2," Roboflow exports, versions 1 and 2, Oct. 2025, CC BY 4.0. [Online]. Available: https://universe.roboflow.com/vision-artificial-k6dcz/nicodogdetection2-ikj95/dataset/1 and https://universe.roboflow.com/vision-artificial-k6dcz/nicodogdetection2-ikj95/dataset/2. Archived exports inspected Oct. 8, 2026.

[8] Ultralytics, "Detection metrics implementation," software v8.3.221, 2025. [Online]. Available: https://github.com/ultralytics/ultralytics/blob/v8.3.221/ultralytics/utils/metrics.py (accessed Oct. 8, 2026).

[9] Ultralytics, "YOLO detection validator," software v8.3.221, 2025. [Online]. Available: https://github.com/ultralytics/ultralytics/blob/v8.3.221/ultralytics/models/yolo/detect/val.py (accessed Oct. 8, 2026).